

VintageCars

Gestion de Voitures de Collection

Django 5

Python 3

SQLite

HTML/CSS

Bootstrap

DJANGO • PYTHON • SQLITE

Application Web de Gestion

Sommaire

1.	Présentation du projet	3
2.	Architecture & Structure	4
3.	Modèle de données (MCD)	5
4.	Fonctionnalités détaillées	6
5.	Gestion des rôles & Sécurité	7
6.	Formulaires & Validation	8
7.	URLs & Navigation	9
8.	Technologies utilisées	10
9.	Instructions de lancement	11

1. Présentation du projet

VintageCars est une application web complète développée avec le framework **Django 5** en Python 3. Elle est destinée à la gestion d'une collection de voitures de collection, permettant de suivre chaque véhicule de son acquisition jusqu'à sa restauration complète.

L'application s'articule autour de deux profils utilisateurs distincts avec des droits différenciés, assurant une séparation claire des responsabilités :

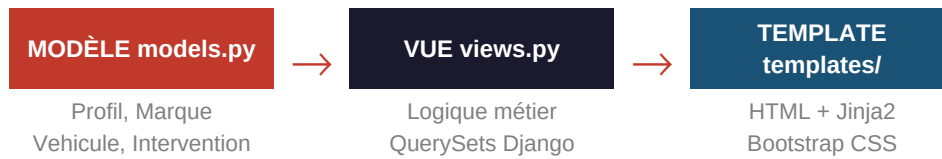
■ Conservateur	■ Mécanicien
• Gérer tous les véhicules • Ajouter / modifier / supprimer • Consulter les statistiques • Gérer les utilisateurs • Accès complet au système	• Consulter les véhicules • Saisir des interventions • Modifier le statut d'un véhicule • Voir les dernières interventions • Accès lecture seule aux stats

Objectifs pédagogiques

- Modélisation MCD → ORM Django
- Système d'authentification et de rôles
- CRUD complet sur les entités
- Formulaires avec validation métier
- Recherche et filtrage avancé
- Statistiques et tableaux de bord

2. Architecture & Structure du projet

Le projet suit l'architecture **MVT (Modèle – Vue – Template)** de Django, avec une séparation claire des responsabilités :



Arborescence des fichiers

```
ProjetVintageCars/
├── .git/                ← Dépôt Git
├── cars/                ← Application principale
│   ├── migrations/     ← Migrations base de données
│   ├── admin.py        ← Interface d'administration
│   ├── apps.py         ← Configuration de l'app
│   ├── forms.py        ← Formulaires Django
│   ├── models.py       ← Modèles de données (ORM)
│   ├── urls.py         ← Routes de l'application
│   └── views.py        ← Logique des vues
├── VintageCars/        ← Configuration du projet
│   ├── settings.py     ← Paramètres Django
│   ├── urls.py         ← URLs racine
│   └── asgi.py / wsgi.py ← Serveurs ASGI/WSGI
├── templates/          ← Templates HTML globaux
│   ├── base.html       ← Template de base (héritage)
│   ├── connexion.html
│   ├── dashboard_conservateur.html
│   ├── dashboard_mecanicien.html
│   ├── vehicules.html
│   ├── detail_vehicule.html
│   ├── vehicule_form.html
│   ├── intervention_form.html
│   ├── modifier_statut.html
│   ├── taux_restauracion.html
│   └── utilisateurs.html
├── vintagecars.db      ← Base de données SQLite
├── manage.py           ← Commandes Django
└── README.md           ← Documentation
```

3. Modèle de données

Le projet comporte **4 modèles** reliés entre eux par des clés étrangères. Voici le détail de chaque entité :

Profil	Type	Description
utilisateur	OneToOneField(Use r)	Lien vers l'utilisateur Django natif
role	CharField	Choix : conservateur mécanicien

Marque	Type	Description
nom_marque	CharField(100)	Nom de la marque (ex: Ferrari)
pays_origine	CharField(100)	Pays de fabrication

Vehicule	Type	Description
marque	ForeignKey(Marque)	Marque du véhicule (CASCADE)
modele	CharField(150)	Modèle du véhicule
annee	IntegerField	Année de fabrication (1900–1999)
chassis	CharField(17, unique)	Numéro VIN unique (17 car.)
statut	CharField(choices)	en_restauration restaure en_attente a_vendre
cylindres	IntegerField	Nombre de cylindres
cylindree	IntegerField	Cylindrée en cm ³

Intervention	Type	Description
vehicule	ForeignKey(Vehicu le)	Véhicule concerné (CASCADE)
type	CharField(200)	Type d'intervention
date	DateField	Date de l'intervention
cout	DecimalField(10, 2)	Coût en MAD

4. Fonctionnalités détaillées

	Authentification Système de connexion sécurisé basé sur l'authentification Django native. Redirection automatique vers le dashboard selon le rôle de l'utilisateur connecté.
	Gestion des véhicules CRUD complet : enregistrement d'un nouveau véhicule, consultation de la fiche technique, modification et suppression (conservateur uniquement). Chaque fiche contient : marque, modèle, année, châssis VIN, statut, cylindres et cylindrée.
	Recherche & Filtrage Moteur de recherche multicritères : texte libre (modèle, marque, châssis), filtre par marque, filtre par statut, fourchette d'années min/max.
	Interventions mécaniques Saisie des interventions avec type, date et coût. Historique complet par véhicule avec cumul des coûts. Accessible aux mécaniciens et conservateurs.
	Taux de restauration Tableau de bord statistique : taux global de restauration (%), coût total des interventions, classement des véhicules par coût, statistiques par marque.
	Gestion utilisateurs Le conservateur peut modifier le rôle de chaque utilisateur directement depuis l'interface. Affichage de la liste complète des profils enregistrés.

5. Gestion des rôles & Sécurité

Chaque vue est protégée par le décorateur `@login_required`. Un contrôle de rôle manuel est effectué au début de chaque vue sensible :

```
from django.contrib.auth.decorators import login_required
@login_required(login_url='connexion') def enregistrer_vehicule(request): if
request.user.profil.role != 'conservateur': messages.error(request, "Accès réservé
au conservateur.") return redirect('dashboard') ...
```

Matrice des droits d'accès

Fonctionnalité	Conservateur	Mécanicien
Se connecter / déconnecter	■	■
Voir la liste des véhicules	■	■
Voir le détail d'un véhicule	■	■
Enregistrer un véhicule	■	■
Modifier la fiche technique	■	■
Supprimer un véhicule	■	■
Modifier le statut	■	■
Saisir une intervention	■	■
Voir le taux de restauration	■	■
Gérer les utilisateurs	■	■

6. Formulaires & Validation

L'application contient **5 formulaires Django** avec validation métier :

Formulaire	Modèle lié	Champs	Validation spéciale
ConnexionForm	—	username, password	Héritage forms.Form
VehiculeForm	Vehicule	marque, modele, annee, chassis, statut, cylindres, cylindree	annee ∈ [1900–1999] chassis = 17 car. (VIN) chassis.upper()
StatutForm	Vehicule	statut	Modification isolée du statut
InterventionForm	Intervention	vehicule, type, date, cout	DateTimeInput type=date
RechercheForm	—	q, marque, statut, annee_min, annee_max	Tous champs optionnels Filtrage OR/AND sur QuerySet

Exemple de validation (VehiculeForm) :

```
def clean_annee(self): annee = self.cleaned_data.get('annee') if annee and (annee < 1900 or annee > 1999): raise forms.ValidationError( "Seuls les véhicules vintage (1900–1999) sont acceptés.") return annee def clean_chassis(self): chassis = self.cleaned_data.get('chassis') if chassis and len(chassis) != 17: raise forms.ValidationError( "Le numéro VIN doit contenir exactement 17 caractères.") return chassis.upper()
```


7. URLs & Navigation

URL	Nom	Vue	Accès
/connexion/	connexion	connexion()	Public
/deconnexion/	deconnexion	deconnexion()	Connecté
/	dashboard	dashboard()	Connecté
/conservateur/dashboard/	dashboard_conservateur	dashboard_conservateur()	Conservateur
/mecanicien/dashboard/	dashboard_mecanicien	dashboard_mecanicien()	Mécanicien
/vehicules/	vehicules	vehicules()	Connecté
/vehicules/nouveau/	enregistrer_vehicule	enregistrer_vehicule()	Conservateur
/vehicules/<pk>/	detail_vehicule	detail_vehicule(pk)	Connecté
/vehicules/<pk>/modifier/	modifier_vehicule	modifier_vehicule(pk)	Conservateur
/vehicules/<pk>/supprimer/	supprimer_vehicule	supprimer_vehicule(pk)	Conservateur
/vehicules/<pk>/statut/	modifier_statut	modifier_statut(pk)	Connecté
/interventions/nouvelle/	saisir_intervention	saisir_intervention()	Connecté
/vehicules/<pk>/intervention/	saisir_intervention_vehicule	saisir_intervention(pk)	Connecté
/restauration/	taux_restauration	taux_restauration()	Connecté
/utilisateurs/	gerer_utilisateurs	gerer_utilisateurs()	Conservateur

8. Technologies utilisées

Django 5.x

Framework web Python

MVT, ORM, authentification, messages, décorateurs, formulaires, migrations

Python 3

Langage principal

POO, QuerySets, annotations, agrégations (Sum, Count)

SQLite

Base de données

Légère, embarquée, parfaite pour le développement et la présentation

HTML5 / CSS3

Interface utilisateur

Templates Django avec héritage (base.html), structure sémantique

Bootstrap

Framework CSS

Grille responsive, composants (cards, tables, forms, badges, modals)

Git

Versionnement

Dépôt local avec historique des commits, branches main

9. Instructions de lancement

- 1 Cloner ou extraire le projet**
Extraire l'archive et se placer dans le dossier `ProjetVintageCars/`
- 2 Installer Django**
`pip install django`
- 3 Appliquer les migrations**
`python manage.py migrate`
- 4 Créer un super-utilisateur (optionnel)**
`python manage.py createsuperuser`
- 5 Lancer le serveur de développement**
`python manage.py runserver`
- 6 Ouvrir dans le navigateur**
`http://127.0.0.1:8000/`

Comptes de test recommandés

Utilisateur	Mot de passe	Rôle	Accès
admin	admin123	Conservateur	Complet
mecanicien1	mec123	Mécanicien	Limité

Projet réalisé dans le cadre du cours de Développement Web — 2025/2026